atividade-curta-3

April 6, 2026

[12]:
```
%matplotlib inline

import matplotlib.pyplot as plt

import cv2
import numpy as np

from tqdm import trange
import tqdm
import random

import scipy.io

from sklearn.cluster import MiniBatchKMeans
from sklearn.neighbors import NearestNeighbors
from kmodes.kmodes import KModes

import math

SEED = 95667
random.seed(SEED)
np.random.seed(SEED)
```

0.1 Image display

[13]:
```
def show_image_and_keypoints( image , kps ) :
    cv2.drawKeypoints( image, kps, image, flags=cv2.
 ↪DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS )

    plt.figure(figsize = (10,10))
    plt.imshow(image, aspect='auto')
    plt.axis('off')
    plt.title('Keypoints and descriptors.')
    plt.show()
```

[14]:
```
def show_top_images_grid(dataset_path, indices, id_test, ids, labels, cols=4):
    """
```

```
    Fetches the query image and its top matches, displaying them in a grid.
    cols: Number of columns you want in the grid.
    """
    images_to_show = []
    titles = []

    # 1. Fetch the Query Image
    label = (ids[id_test] - 1) // 80
    name = f"{dataset_path}/jpg/{label}/image_{str(ids[id_test]).zfill(4)}.jpg"

    image = cv2.imread(name)
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    images_to_show.append(image)
    titles.append(f"Query Label: {labels[id_test]}\n(ID: {ids[id_test]})")

    indice = 0
    # 2. Fetch the Match Images from indices
    for i in indices[0]:
        label_i = labels[i]
        name = f"{dataset_path}/jpg/{label_i}/image_{str(ids[i]).zfill(4)}.jpg"

        image = cv2.imread(name)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

        images_to_show.append(image)
        titles.append(f"{indice}° Match: ID {ids[i]} \n Label: {label_i} -␣
↪{'Correct' if label_i == labels[id_test] else 'Incorrect'}")
        indice += 1

    # Create the Figure and Subplots Header
    # Adjust figsize based on your layout preference (width, height)
    fig_header, axes_header = plt.subplots(1, 2, figsize=(20, 4))

    axes_header[0].imshow(images_to_show[0])
    axes_header[0].set_title(f'Query. \n Label: {labels[id_test]}\n(ID:␣
↪{ids[id_test]})')
    axes_header[0].axis('off') # Hide the axis ticks and borders

    axes_header[1].imshow(images_to_show[1])
    axes_header[1].set_title(f'Sanity Test. \n Label:␣
↪{labels[indices[0][0]]}\n(ID: {ids[indices[0][0]]})')
    axes_header[1].axis('off') # Hide the axis ticks and borders


    # 3. Calculate Grid Dimensions
    n_images = len(images_to_show) - 2
```

```
    rows = math.ceil(n_images / cols)

    # 4. Create the Figure and Subplots
    # Adjust figsize based on your layout preference (width, height)
    fig, axes = plt.subplots(rows, cols, figsize=(15, 4 * rows))

    # Flatten axes array to easily iterate over it, even if it's a 2D grid
    if n_images > 1:
        axes = axes.flatten()
    else:
        axes = [axes] # Handle edge case where there's only 1 image

    # 5. Populate the Grid
    for idx in range(len(axes)):
        if idx < n_images:
            axes[idx].imshow(images_to_show[idx+2])
            axes[idx].set_title(titles[idx+2])
            axes[idx].axis('off') # Hide the axis ticks and borders
        else:
            # Turn off empty subplots if the grid isn't perfectly filled
            axes[idx].axis('off')

    plt.tight_layout()
    plt.show()
```

0.2 Vocabulary Creation Steps

detectar e descrever os keypoints de todas as imagens a junção de todos os descritores cria o vocabulário

[15]:
```
def random_detect_and_describe_keypoints( image ):
    kps_random = []

    img_height = image.shape[0]
    img_width = image.shape[1]

    for i in range(300):
        x = random.randint(0, img_width - 1)
        y = random.randint(0, img_height - 1)
        kps_random.append(cv2.KeyPoint(x, y, 10))

    return kps_random
```

[16]:
```
def grid_detect_and_describe_keypoints( image ):
    kps_grid = []

    cell_height = image.shape[0] / 20
```

```
    cell_width = image.shape[1] / 20

    for i in range(20):
        for j in range(20):
            x = int((i + 0.5) * cell_width)
            y = int((j + 0.5) * cell_height)
            kps_grid.append(cv2.KeyPoint(x, y, 10))

    return kps_grid
```

[17]:
```
def detect_and_describe_keypoints ( image, algorithm='orb' ) :

    image_gray = cv2.cvtColor( image , cv2.COLOR_BGR2GRAY )

    # criação do objeto de detecção e descrição de acordo com o algoritmo␣
 ↪escolhido
    if algorithm in ['sift', 'random-sift', 'grid-sift']:
        keypoint = cv2.SIFT_create()
    elif algorithm in ['orb', 'orb-hamming', 'random-orb', 'grid-orb'] :
        keypoint = cv2.ORB_create()
    else :
        print('Error: algorithm not defined')
        return None

    # detecção de keypoints
    if algorithm in ['random-orb','random-sift']:
        kps = random_detect_and_describe_keypoints(image_gray)
    elif algorithm in ['grid-orb','grid-sift']:
        kps = grid_detect_and_describe_keypoints(image_gray)
    else:
        kps = keypoint.detect(image_gray, None)

    # descreve os keypoints
    kps, descs = keypoint.compute( image_gray, kps )

    return kps, descs
```

[18]:
```
def create_vocabulary ( dataset_path, algorithm='orb', show_image=False,␣
 ↪debug=False ) :
    mat = scipy.io.loadmat( dataset_path+'/datasplits.mat' )

    ids = mat['trn1'][0] #  'val1' or 'tst1'

    # criação do array de descritores de acordo com o algoritmo escolhido
    if algorithm == 'orb-hamming':
        train_descs = np.ndarray( shape=(0,32) , dtype=np.uint8 ) # garante␣
 ↪binário
```

```
    elif algorithm in ['orb', 'random-orb', 'grid-orb']:
        train_descs = np.ndarray( shape=(0,32) , dtype=float )
    elif algorithm in ['sift', 'random-sift', 'grid-sift']:
        train_descs = np.ndarray( shape=(0,128) , dtype=float )
    else :
        print('Error:Algorithm not defined.')
        return None

    for id in tqdm.tqdm(ids, desc='Processing train set') :

        label = (id - 1) // 80
        name = dataset_path + '/jpg/' + str(label) + '/image_' + str(id).
↪zfill(4) + '.jpg'

        image = cv2.imread( name )

        if image is None:
            print(f'Reading image Error. Path: {name}')
            return None

        kps, descs = detect_and_describe_keypoints ( image, algorithm=algorithm␣
↪)

        if algorithm == 'orb-hamming':
            descs = descs.astype(np.uint8)
        else:
            descs = descs.astype(np.float32)

        train_descs = np.concatenate((train_descs, descs), axis=0)

        if show_image :
            show_image_and_keypoints(image, kps)

        if debug :
            print( name )
            print( 'Number of keypoints: ', len(kps) )
            print( 'Number of images: ', len(ids) )
            print( 'Descriptor size: ', len(descs[0]) )
            print( type(descs[0]) )

    print( ' -> [I] Image Loader Info:\n',
      '\nTrain len: ', len(train_descs),
      '\nNumber of images: ', len(ids),
      '\nDescriptor size: ', len(descs[0])
      )

    return train_descs
```

0.3 Create Dictionary

faz a clusterização dos descritores (vocabulário) os centroides representam o dicionário

[19]:
```
def create_dictionary_kmeans ( vocabulary, num_cluster, algorithm='sift' ) :

    print( ' -> [I] Dictionary Info:\n',
        '\nTrain len: ', len(vocabulary),
        '\nDimension: ', len(vocabulary[0]),
        '\nClusters: ', num_cluster
        )

    # se o algoritmo for orb-hamming, usamos o KModes com a distancia de Hamming
    if algorithm == 'orb-hamming':
        if vocabulary.dtype != np.uint8:
            vocabulary = vocabulary.astype(np.uint8)
        dictionary = KModes( n_clusters=num_cluster, init='Huang', n_init=1, ␣
↪max_iter=50, verbose=0 )
    else:
        dictionary = MiniBatchKMeans( n_clusters=num_cluster, batch_size=1000,␣
↪random_state=SEED )

    print ( 'Learning dictionary by Kmeans...')
    # clusteriza o vocabulary
    # retorna o dictionary treinado
    dictionary = dictionary.fit( vocabulary )

    # dictionary.cluster_centers_  sao os centroides
    print ( 'Done.')

    return dictionary
```

0.4 Image Representation

cria o histograma que representa a imagem

[21]:
```
def create_bovw_descriptors (image, dictionary, algorithm='orb') :

    descs = detect_and_describe_keypoints( image, algorithm=algorithm )[1] #␣
↪retorna os keypoints e seus descritores

    if algorithm == 'orb-hamming':
        predicted = dictionary.predict(descs.astype(np.uint8))
        hist = np.histogram(predicted, bins=range(0, dictionary.n_clusters +␣
↪1))[0]
        desc_bovw = (hist > 0).astype(np.uint8)
    else:
        if descs.dtype != np.double:
```

```
            descs = descs.astype(np.double)
        predicted = dictionary.predict(np.array(descs, dtype=np.double))
        desc_bovw = np.histogram(predicted, bins=range(0, dictionary.
↪n_clusters+1))[0]

    return desc_bovw
```

0.5 Describe Dataset Images

cria um descritor para cada imagem do dataset

[22]:
```
def represent_dataset( dataset_path, dictionary, test=False, algorithm='orb' ) :

    mat = scipy.io.loadmat( dataset_path+'/datasplits.mat' )

    if test :
        ids = mat['tst1'][0] #  'tst1' or 'trn1' or 'val1'
    else :
        ids = mat['val1'][0] #  'tst1' or 'trn1' or 'val1'

    space = []
    labels = []

    for id in tqdm.tqdm(ids, desc='Processing test set' if test else␣
↪'Processing val set') :

        label = (id - 1) // 80
        name = dataset_path + '/jpg/' + str(label) + '/image_' + str(id).
↪zfill(4) + '.jpg'

        image = cv2.imread( name )

        if image is None:
            print(f'Reading image Error. Path: {name}')
            return None

        desc_bovw = create_bovw_descriptors(image, dictionary,␣
↪algorithm=algorithm)

        space.append(desc_bovw)
        labels.append(label)

    print( ' -> [I] Space Describing Info:\n',
        '\nNumber of images: ', len(space),
        '\nNumber of labels: ', len(labels),
        '\nDimension: ', len(space[0])
        )
```

```
    return space , labels
```

0.6 Acurracy

mede a precisão do image retrieval

[23]:
```
def run_experiment ( space , labels , dictionary , dataset_path, test=False,␣
↪algorithm='orb', top=10 ) :
    if algorithm == 'orb-hamming':
        knn = NearestNeighbors(n_neighbors=top+1, metric='hamming').fit(space)
    else:
        knn = NearestNeighbors(n_neighbors=top+1, metric='euclidean').fit(space)

    mat = scipy.io.loadmat( dataset_path+'/datasplits.mat' )

    if test :
        ids = mat['tst1'][0] #  'tst1' or 'trn1' or 'val1'
    else :
        ids = mat['val1'][0] #  'tst1' or 'trn1' or 'val1'

    accuracy_t = 0

    for id_test in tqdm.tqdm(ids, desc= 'running the test phase' if test else␣
↪'running the val phase') :

        label = (id_test - 1) // 80
        name = dataset_path + '/jpg/' + str(label) + '/image_' + str(id_test).
↪zfill(4) + '.jpg'

        image = cv2.imread( name )

        desc_bovw = create_bovw_descriptors(image, dictionary,␣
↪algorithm=algorithm)

        indices = knn.kneighbors(desc_bovw.reshape(1, -1))[1] #retorna as␣
↪imagens mais parecidas

        labels_top = [ labels[i] for i in indices[0] ]

        accuracy = sum( np.equal(labels_top, label) )
        accuracy =( (accuracy-1)/(top) ) * 100
        accuracy_t = accuracy_t + accuracy

    avg_acc = accuracy_t / len(ids)
```

```
    print(f'Average accuracy in the', 'test' if test else 'validation', f'set:␣
↪{avg_acc:5.2f}%')

    return avg_acc
```

0.7 Image Retrieval

as k imagens mais proximas

[26]:
```
def retrieve_single_image ( space , labels , dictionary , dataset_path,␣
↪algorithm='orb', top=10 ) :
    if algorithm == 'orb-hamming':
        knn = NearestNeighbors(n_neighbors=top+1, metric='hamming').fit(space)
    else:
        knn = NearestNeighbors(n_neighbors=top+1, metric='euclidean').fit(space)


    mat = scipy.io.loadmat( dataset_path+'/datasplits.mat' )

    ids = mat['tst1'][0] #  'trn1' or 'val1'

    id_test = random.randrange( len(ids) )

    label = (ids[id_test] - 1) // 80
    name = dataset_path + '/jpg/' + str(label) + '/image_' + str(ids[id_test]).
↪zfill(4) + '.jpg'

    image = cv2.imread( name )

    if image is None:
        print(f'Reading image Error. Path: {name}')
        return None

    desc_bovw = create_bovw_descriptors(image, dictionary, algorithm=algorithm)

    distances, indices = knn.kneighbors(desc_bovw.reshape(1, -1))

    # show_top_images(dataset_path, indices, id_test, ids, labels)
    show_top_images_grid(dataset_path, indices, id_test, ids, labels)

    labels_top = [ int(labels[i]) for i in indices[0] ]

    accuracy = sum( np.equal( label , labels_top ) )
    accuracy =( (accuracy-1)/(top) ) * 100

    print(f'Accuracy for image id {ids[id_test]}: {accuracy:5.2f}%')
```

```
    print(name)
    print(f'Image: {ids[id_test]} with label {labels[id_test]}')
    print(f'Closest image: {ids[indices[0][0]]} with distance {distances[0][0]}␣
 ↪and label {labels[indices[0][0]]}')
    print('Distances: ',distances)
    print('Indices: ',indices[0])
    print('Labels: ',labels_top)
```

0.8 Results

[13]:
```
import pandas as pd
from IPython.display import display
# destaca a melhor acurácia em cada linha da tabela
def highlight_max(row):
    numeric_cols = row[1:]
    max_val = numeric_cols.max()
    return ['' if col == "Dic. Size" else
            'background-color: green' if row[col] == max_val else ''
            for col in row.index]

def show_results_table(results):
    # separa os resultados de BoVW dos resultados de LBP
    bovw_results = {k: v for k, v in results.items() if k != 'LBP'}

    if bovw_results:
        # mostra os resultados dos algoritmos BoVW com diferentes tamanhos de␣
 ↪dicionário
        df_bovw = pd.DataFrame(bovw_results)
        df_bovw.index.name = "Dic. Size"
        df_bovw = df_bovw.reset_index()

        # converte os valores do índice para inteiro
        df_bovw["Dic. Size"] = df_bovw["Dic. Size"].astype(int)

        df_bovw = df_bovw.round(2)

        print("\n" + "="*70)
        print("TABELA 1: BoVW Algorithms (different dictionary sizes)")
        print("="*70)
        display(df_bovw)

        # encontra a melhor configuração global entre todos os algoritmos BoVW
        best = None
        best_val = 0

        for alg in bovw_results:
            for k in bovw_results[alg]:
```

```
                if bovw_results[alg][k] > best_val:
                    best_val = bovw_results[alg][k]
                    best = (alg, k)

        if best:
            print(f"\nBest BoVW configuration: Algorithm = {best[0].upper()}, K␣
↪= {best[1]} → {best_val:.2f}%")

    # mostra em uma tabela separada para o LBP
    if 'LBP' in results and results['LBP']:
        print("\n" + "="*70)
        print("TABELA 2: LBP (global descriptor)")
        print("="*70)

        # LBP não usa variação de tamanho de dicionário, então apenas extrai o␣
↪valor único
        lbp_acc = results['LBP'].get('N/A', list(results['LBP'].values())[0] if␣
↪results['LBP'] else 0)

        df_lbp = pd.DataFrame({
            'Algorithm': ['LBP'],
            'Accuracy (%)': [round(lbp_acc, 2)]
        })

        display(df_lbp)
        print(f"\nLBP Average Accuracy: {lbp_acc:.2f}%")
```

0.9 LBP

[14]:
```
from skimage.feature import local_binary_pattern

# extrai o histograma da imagem
def extract_lbp(image_gray, radius=1, n_points=8):
    lbp = local_binary_pattern(image_gray, n_points, radius, method='uniform')

    n_bins = n_points + 2
    hist, _ = np.histogram(lbp.ravel(), bins=n_bins, range=(0, n_bins))

    # normaliza o histograma para float
    hist = hist.astype("float")
    hist /= hist.sum()

    return hist

# representa o dataset usando o histograma LBP
def represent_dataset_lbp(dataset_path, test=False):
    mat = scipy.io.loadmat(dataset_path+'/datasplits.mat')
```

```
    if test:
        ids = mat['tst1'][0]
    else:
        ids = mat['val1'][0]

    space = []
    labels = []

    for id in tqdm.tqdm(ids, desc='Processing LBP'):
        label = (id - 1) // 80
        name = f"{dataset_path}/jpg/{label}/image_{str(id).zfill(4)}.jpg"

        image = cv2.imread(name)
        if image is None:
            print(f'Error: {name}')
            return None

        image_gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

        lbp_hist = extract_lbp(image_gray)

        space.append(lbp_hist)
        labels.append(label)

    return space, labels

def run_experiment_lbp(space, labels, dataset_path, test=False, top=10):
    knn = NearestNeighbors(n_neighbors=top+1, metric='euclidean').fit(space)

    mat = scipy.io.loadmat(dataset_path+'/datasplits.mat')

    if test:
        ids = mat['tst1'][0]
    else:
        ids = mat['val1'][0]

    accuracy_t = 0

    for id_test in tqdm.tqdm(ids, desc='running LBP'):
        label = (id_test - 1) // 80
        name = f"{dataset_path}/jpg/{label}/image_{str(id_test).zfill(4)}.jpg"

        image = cv2.imread(name)
        image_gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
        lbp_hist = extract_lbp(image_gray)
```

```
        # busca as imagens mais próximas no descriptor LBP
        indices = knn.kneighbors(lbp_hist.reshape(1, -1))[1]
        labels_top = [labels[i] for i in indices[0]]

        accuracy = sum(np.equal(labels_top, label))
        accuracy = ((accuracy-1)/(top)) * 100
        accuracy_t += accuracy

    avg_acc = accuracy_t / len(ids)
    return avg_acc
```

0.10 Tune

[15]:
```
algorithms = ["random-orb", "grid-orb", "orb", "orb-hamming", "sift",␣
 ↪"random-sift", "grid-sift"]
clusters = [100, 200, 400, 600, 800]
dataset_path = '/Users/nicholasbarbosa/Mestrado/inf692/flowers_classes'

results = {alg: {} for alg in algorithms}
results["LBP"] = {}  # LBP não usa clusters


# roda LBP separadamente
print("\n" + "="*50)
print("ALGORITHM: LBP")
print("="*50)

space_lbp, labels_lbp = represent_dataset_lbp(dataset_path, test=False)
acc_lbp = run_experiment_lbp(space_lbp, labels_lbp, dataset_path, test=False)
results["LBP"]["N/A"] = acc_lbp
print(f"Average accuracy (LBP): {acc_lbp:.2f}%")

for algorithm in algorithms:
    print("\n" + "="*50)
    print(f"ALGORITHM: {algorithm.upper()}")
    print("="*50)

    print("-"*50)
    print("\nStep 1: Vocabulary Creation")
    print("-"*50)
    vocabulary = create_vocabulary(dataset_path, algorithm=algorithm)

    for num_cluster in clusters:
        print("\n" + "-"*50)
        print(f"Step 2: Dictionary Creation | K = {num_cluster}")
        print("-"*50)
```

```
        dictionary = create_dictionary_kmeans(vocabulary,␣
↪num_cluster=num_cluster, algorithm=algorithm)

        print("\nStep 3: Represent Dataset (BoVW)")
        print("-"*50)
        space, labels = represent_dataset(dataset_path, dictionary,␣
↪algorithm=algorithm)

        print("\nStep 4: Running Experiment (Validation)")
        print("-"*50)
        acc = run_experiment(space, labels, dictionary, dataset_path,␣
↪algorithm=algorithm)

        # salva resultado
        results[algorithm][num_cluster] = acc
```

```
==================================================
ALGORITHM: LBP
==================================================

Processing LBP:   0%|          | 0/340 [00:00<?, ?it/s]

Processing LBP: 100%|     | 340/340 [00:05<00:00, 62.81it/s]
running LBP: 100%|     | 340/340 [00:05<00:00, 63.67it/s]

Average accuracy (LBP): 7.85%

==================================================
ALGORITHM: RANDOM-ORB
==================================================
--------------------------------------------------

Step 1: Vocabulary Creation
--------------------------------------------------

Processing train set: 100%|     | 680/680 [00:04<00:00, 162.84it/s]

 -> [I] Image Loader Info:

Train len:  161198
Number of images:  680
Descriptor size:  32

--------------------------------------------------
Step 2: Dictionary Creation | K = 100
--------------------------------------------------
 -> [I] Dictionary Info:

Train len:  161198
```

```
Dimension:  32
Clusters:  100
Learning dictionary by Kmeans…
Done.

Step 3: Represent Dataset (BoVW)
--------------------------------------------------

Processing val set: 100%|      | 340/340 [00:00<00:00, 507.61it/s]

 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  100

Step 4: Running Experiment (Validation)
--------------------------------------------------

running the val phase: 100%|      | 340/340 [00:00<00:00, 498.21it/s]

Average accuracy in the validation set:  3.56%

--------------------------------------------------
Step 2: Dictionary Creation | K = 200
--------------------------------------------------
 -> [I] Dictionary Info:

Train len:  161198
Dimension:  32
Clusters:  200
Learning dictionary by Kmeans…
Done.

Step 3: Represent Dataset (BoVW)
--------------------------------------------------

Processing val set: 100%|      | 340/340 [00:00<00:00, 577.49it/s]

 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  200

Step 4: Running Experiment (Validation)
--------------------------------------------------

running the val phase: 100%|      | 340/340 [00:00<00:00, 500.08it/s]

Average accuracy in the validation set:  3.26%
```

```
--------------------------------------------------
Step 2: Dictionary Creation | K = 400
--------------------------------------------------
 -> [I] Dictionary Info:

Train len:  161198
Dimension:  32
Clusters:  400
Learning dictionary by Kmeans…
Done.

Step 3: Represent Dataset (BoVW)
--------------------------------------------------

Processing val set: 100%|     | 340/340 [00:00<00:00, 541.35it/s]

 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  400

Step 4: Running Experiment (Validation)
--------------------------------------------------

running the val phase: 100%|     | 340/340 [00:00<00:00, 428.66it/s]

Average accuracy in the validation set:  2.71%

--------------------------------------------------
Step 2: Dictionary Creation | K = 600
--------------------------------------------------
 -> [I] Dictionary Info:

Train len:  161198
Dimension:  32
Clusters:  600
Learning dictionary by Kmeans…
Done.

Step 3: Represent Dataset (BoVW)
--------------------------------------------------

Processing val set: 100%|     | 340/340 [00:00<00:00, 524.79it/s]

 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  600
```

```
Step 4: Running Experiment (Validation)
--------------------------------------------------

running the val phase: 100%|      | 340/340 [00:00<00:00, 432.93it/s]

Average accuracy in the validation set:  2.15%

--------------------------------------------------
Step 2: Dictionary Creation | K = 800
--------------------------------------------------
 -> [I] Dictionary Info:

Train len:  161198
Dimension:  32
Clusters:  800
Learning dictionary by Kmeans…
Done.

Step 3: Represent Dataset (BoVW)
--------------------------------------------------

Processing val set: 100%|      | 340/340 [00:00<00:00, 470.70it/s]

 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  800

Step 4: Running Experiment (Validation)
--------------------------------------------------

running the val phase: 100%|      | 340/340 [00:00<00:00, 400.17it/s]

Average accuracy in the validation set:  1.71%

==================================================
ALGORITHM: GRID-ORB
==================================================
--------------------------------------------------

Step 1: Vocabulary Creation
--------------------------------------------------

Processing train set: 100%|      | 680/680 [00:05<00:00, 122.59it/s]

 -> [I] Image Loader Info:

Train len:  220320
Number of images:  680
Descriptor size:  32
```

```
--------------------------------------------------
Step 2: Dictionary Creation | K = 100
--------------------------------------------------
 -> [I] Dictionary Info:

Train len:  220320
Dimension:  32
Clusters:  100
Learning dictionary by Kmeans…
Done.

Step 3: Represent Dataset (BoVW)
--------------------------------------------------

Processing val set: 100%|     | 340/340 [00:01<00:00, 312.21it/s]

 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  100

Step 4: Running Experiment (Validation)
--------------------------------------------------

running the val phase: 100%|     | 340/340 [00:00<00:00, 487.05it/s]

Average accuracy in the validation set:  9.41%

--------------------------------------------------
Step 2: Dictionary Creation | K = 200
--------------------------------------------------
 -> [I] Dictionary Info:

Train len:  220320
Dimension:  32
Clusters:  200
Learning dictionary by Kmeans…
Done.

Step 3: Represent Dataset (BoVW)
--------------------------------------------------

Processing val set: 100%|     | 340/340 [00:00<00:00, 472.29it/s]

 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  200
```

```
Step 4: Running Experiment (Validation)
-------------------------------------------------

running the val phase: 100%|      | 340/340 [00:00<00:00, 478.75it/s]

Average accuracy in the validation set:  8.62%

-------------------------------------------------
Step 2: Dictionary Creation | K = 400
-------------------------------------------------
 -> [I] Dictionary Info:

Train len:  220320
Dimension:  32
Clusters:  400
Learning dictionary by Kmeans…
Done.

Step 3: Represent Dataset (BoVW)
-------------------------------------------------

Processing val set: 100%|      | 340/340 [00:00<00:00, 466.85it/s]

 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  400

Step 4: Running Experiment (Validation)
-------------------------------------------------

running the val phase: 100%|      | 340/340 [00:01<00:00, 328.98it/s]

Average accuracy in the validation set:  8.79%

-------------------------------------------------
Step 2: Dictionary Creation | K = 600
-------------------------------------------------
 -> [I] Dictionary Info:

Train len:  220320
Dimension:  32
Clusters:  600
Learning dictionary by Kmeans…
Done.

Step 3: Represent Dataset (BoVW)
-------------------------------------------------

Processing val set: 100%|      | 340/340 [00:00<00:00, 511.50it/s]
```

```
 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  600

Step 4: Running Experiment (Validation)
-------------------------------------------------

running the val phase: 100%|      | 340/340 [00:00<00:00, 437.06it/s]

Average accuracy in the validation set:  8.26%

-------------------------------------------------
Step 2: Dictionary Creation | K = 800
-------------------------------------------------
 -> [I] Dictionary Info:

Train len:  220320
Dimension:  32
Clusters:  800
Learning dictionary by Kmeans…
Done.

Step 3: Represent Dataset (BoVW)
-------------------------------------------------

Processing val set: 100%|      | 340/340 [00:00<00:00, 485.20it/s]

 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  800

Step 4: Running Experiment (Validation)
-------------------------------------------------

running the val phase: 100%|      | 340/340 [00:00<00:00, 401.23it/s]

Average accuracy in the validation set:  9.47%

=================================================
ALGORITHM: ORB
=================================================
-------------------------------------------------

Step 1: Vocabulary Creation
-------------------------------------------------

Processing train set: 100%|      | 680/680 [00:11<00:00, 57.69it/s]
```

```
 -> [I] Image Loader Info:

Train len:  333473
Number of images:  680
Descriptor size:  32

--------------------------------------------------
Step 2: Dictionary Creation | K = 100
--------------------------------------------------
 -> [I] Dictionary Info:

Train len:  333473
Dimension:  32
Clusters:  100
Learning dictionary by Kmeans…
Done.

Step 3: Represent Dataset (BoVW)
--------------------------------------------------

Processing val set: 100%|     | 340/340 [00:02<00:00, 131.60it/s]
 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  100

Step 4: Running Experiment (Validation)
--------------------------------------------------

running the val phase: 100%|     | 340/340 [00:02<00:00, 138.00it/s]

Average accuracy in the validation set: 15.76%

--------------------------------------------------
Step 2: Dictionary Creation | K = 200
--------------------------------------------------
 -> [I] Dictionary Info:

Train len:  333473
Dimension:  32
Clusters:  200
Learning dictionary by Kmeans…
Done.

Step 3: Represent Dataset (BoVW)
--------------------------------------------------

Processing val set: 100%|     | 340/340 [00:02<00:00, 129.39it/s]
```

```
 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  200

Step 4: Running Experiment (Validation)
-------------------------------------------------

running the val phase: 100%|      | 340/340 [00:02<00:00, 136.43it/s]

Average accuracy in the validation set: 15.56%

-------------------------------------------------
Step 2: Dictionary Creation | K = 400
-------------------------------------------------
 -> [I] Dictionary Info:

Train len:  333473
Dimension:  32
Clusters:  400
Learning dictionary by Kmeans…
Done.

Step 3: Represent Dataset (BoVW)
-------------------------------------------------

Processing val set: 100%|      | 340/340 [00:02<00:00, 137.78it/s]

 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  400

Step 4: Running Experiment (Validation)
-------------------------------------------------

running the val phase: 100%|      | 340/340 [00:02<00:00, 126.63it/s]

Average accuracy in the validation set: 15.35%

-------------------------------------------------
Step 2: Dictionary Creation | K = 600
-------------------------------------------------
 -> [I] Dictionary Info:

Train len:  333473
Dimension:  32
Clusters:  600
Learning dictionary by Kmeans…
```

```
Done.

Step 3: Represent Dataset (BoVW)
-------------------------------------------------

Processing val set: 100%|     | 340/340 [00:02<00:00, 121.65it/s]

 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  600

Step 4: Running Experiment (Validation)
-------------------------------------------------

running the val phase: 100%|     | 340/340 [00:02<00:00, 126.85it/s]

Average accuracy in the validation set: 14.56%

-------------------------------------------------
Step 2: Dictionary Creation | K = 800
-------------------------------------------------
 -> [I] Dictionary Info:

Train len:  333473
Dimension:  32
Clusters:  800
Learning dictionary by Kmeans…
Done.

Step 3: Represent Dataset (BoVW)
-------------------------------------------------

Processing val set: 100%|     | 340/340 [00:02<00:00, 124.01it/s]

 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  800

Step 4: Running Experiment (Validation)
-------------------------------------------------

running the val phase: 100%|     | 340/340 [00:02<00:00, 120.41it/s]

Average accuracy in the validation set: 14.15%

=================================================
ALGORITHM: ORB-HAMMING
=================================================
```

```
-------------------------------------------------

Step 1: Vocabulary Creation
-------------------------------------------------

Processing train set: 100%|      | 680/680 [00:04<00:00, 139.20it/s]
 -> [I] Image Loader Info:

Train len:  333473
Number of images:  680
Descriptor size:  32

-------------------------------------------------
Step 2: Dictionary Creation | K = 100
-------------------------------------------------
 -> [I] Dictionary Info:

Train len:  333473
Dimension:  32
Clusters:  100
Learning dictionary by Kmeans…
Done.

Step 3: Represent Dataset (BoVW)
-------------------------------------------------

Processing val set: 100%|      | 340/340 [00:13<00:00, 26.02it/s]
 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  100

Step 4: Running Experiment (Validation)
-------------------------------------------------

running the val phase: 100%|      | 340/340 [00:12<00:00, 26.33it/s]
Average accuracy in the validation set:  6.74%

-------------------------------------------------
Step 2: Dictionary Creation | K = 200
-------------------------------------------------
 -> [I] Dictionary Info:

Train len:  333473
Dimension:  32
Clusters:  200
Learning dictionary by Kmeans…
```

```
Done.

Step 3: Represent Dataset (BoVW)
-------------------------------------------------

Processing val set: 100%|     | 340/340 [00:22<00:00, 15.34it/s]

 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  200

Step 4: Running Experiment (Validation)
-------------------------------------------------

running the val phase: 100%|     | 340/340 [00:22<00:00, 15.41it/s]

Average accuracy in the validation set:  7.85%

-------------------------------------------------
Step 2: Dictionary Creation | K = 400
-------------------------------------------------
 -> [I] Dictionary Info:

Train len:  333473
Dimension:  32
Clusters:  400
Learning dictionary by Kmeans…
Done.

Step 3: Represent Dataset (BoVW)
-------------------------------------------------

Processing val set: 100%|     | 340/340 [00:40<00:00,  8.46it/s]

 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  400

Step 4: Running Experiment (Validation)
-------------------------------------------------

running the val phase: 100%|     | 340/340 [00:39<00:00,  8.50it/s]

Average accuracy in the validation set: 11.79%

-------------------------------------------------
Step 2: Dictionary Creation | K = 600
-------------------------------------------------
```

```
 -> [I] Dictionary Info:

Train len:  333473
Dimension:  32
Clusters:  600
Learning dictionary by Kmeans…
Done.

Step 3: Represent Dataset (BoVW)
--------------------------------------------------

Processing val set: 100%|     | 340/340 [00:57<00:00,  5.87it/s]

 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  600

Step 4: Running Experiment (Validation)
--------------------------------------------------

running the val phase: 100%|     | 340/340 [00:57<00:00,  5.93it/s]

Average accuracy in the validation set: 13.68%

--------------------------------------------------
Step 2: Dictionary Creation | K = 800
--------------------------------------------------
 -> [I] Dictionary Info:

Train len:  333473
Dimension:  32
Clusters:  800
Learning dictionary by Kmeans…
Done.

Step 3: Represent Dataset (BoVW)
--------------------------------------------------

Processing val set: 100%|     | 340/340 [01:15<00:00,  4.50it/s]

 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  800

Step 4: Running Experiment (Validation)
--------------------------------------------------
```

```
running the val phase: 100%|      | 340/340 [01:15<00:00,  4.50it/s]

Average accuracy in the validation set: 13.09%

================================================
ALGORITHM: SIFT
================================================
------------------------------------------------

Step 1: Vocabulary Creation
------------------------------------------------

Processing train set: 100%|      | 680/680 [06:44<00:00,  1.68it/s]

 -> [I] Image Loader Info:

Train len:  1233815
Number of images:  680
Descriptor size:  128

------------------------------------------------
Step 2: Dictionary Creation | K = 100
------------------------------------------------
 -> [I] Dictionary Info:

Train len:  1233815
Dimension:  128
Clusters:  100
Learning dictionary by Kmeans…
Done.

Step 3: Represent Dataset (BoVW)
------------------------------------------------

Processing val set: 100%|      | 340/340 [00:19<00:00, 17.52it/s]

 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  100

Step 4: Running Experiment (Validation)
------------------------------------------------

running the val phase: 100%|      | 340/340 [00:20<00:00, 16.86it/s]

Average accuracy in the validation set: 19.97%

------------------------------------------------
Step 2: Dictionary Creation | K = 200
```

```
------------------------------------------------
 -> [I] Dictionary Info:

Train len:  1233815
Dimension:  128
Clusters:  200
Learning dictionary by Kmeans…
Done.

Step 3: Represent Dataset (BoVW)
------------------------------------------------

Processing val set: 100%|     | 340/340 [00:18<00:00, 18.15it/s]

 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  200

Step 4: Running Experiment (Validation)
------------------------------------------------

running the val phase: 100%|     | 340/340 [00:18<00:00, 18.39it/s]

Average accuracy in the validation set: 21.38%

------------------------------------------------
Step 2: Dictionary Creation | K = 400
------------------------------------------------
 -> [I] Dictionary Info:

Train len:  1233815
Dimension:  128
Clusters:  400
Learning dictionary by Kmeans…
Done.

Step 3: Represent Dataset (BoVW)
------------------------------------------------

Processing val set: 100%|     | 340/340 [00:18<00:00, 18.02it/s]

 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  400

Step 4: Running Experiment (Validation)
------------------------------------------------
```

```
running the val phase: 100%|      | 340/340 [00:19<00:00, 17.67it/s]
Average accuracy in the validation set: 22.06%

--------------------------------------------------
Step 2: Dictionary Creation | K = 600
--------------------------------------------------
 -> [I] Dictionary Info:

Train len:  1233815
Dimension:  128
Clusters:  600
Learning dictionary by Kmeans…
Done.

Step 3: Represent Dataset (BoVW)
--------------------------------------------------

Processing val set: 100%|      | 340/340 [00:19<00:00, 17.54it/s]
 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  600

Step 4: Running Experiment (Validation)
--------------------------------------------------

running the val phase: 100%|      | 340/340 [00:20<00:00, 16.97it/s]
Average accuracy in the validation set: 21.94%

--------------------------------------------------
Step 2: Dictionary Creation | K = 800
--------------------------------------------------
 -> [I] Dictionary Info:

Train len:  1233815
Dimension:  128
Clusters:  800
Learning dictionary by Kmeans…
Done.

Step 3: Represent Dataset (BoVW)
--------------------------------------------------

Processing val set: 100%|      | 340/340 [00:21<00:00, 15.78it/s]
 -> [I] Space Describing Info:
```

```
Number of images:  340
Number of labels:  340
Dimension:  800

Step 4: Running Experiment (Validation)
-------------------------------------------------

running the val phase: 100%|      | 340/340 [00:22<00:00, 14.88it/s]

Average accuracy in the validation set: 22.09%

=================================================
ALGORITHM: RANDOM-SIFT
=================================================
-------------------------------------------------

Step 1: Vocabulary Creation
-------------------------------------------------

Processing train set: 100%|      | 680/680 [00:30<00:00, 22.28it/s]

 -> [I] Image Loader Info:

Train len:  204000
Number of images:  680
Descriptor size:  128

-------------------------------------------------
Step 2: Dictionary Creation | K = 100
-------------------------------------------------
 -> [I] Dictionary Info:

Train len:  204000
Dimension:  128
Clusters:  100
Learning dictionary by Kmeans…
Done.

Step 3: Represent Dataset (BoVW)
-------------------------------------------------

Processing val set: 100%|      | 340/340 [00:04<00:00, 80.17it/s]

 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  100

Step 4: Running Experiment (Validation)
-------------------------------------------------
```

```
running the val phase: 100%|      | 340/340 [00:03<00:00, 101.54it/s]
Average accuracy in the validation set: 14.71%

--------------------------------------------------
Step 2: Dictionary Creation | K = 200
--------------------------------------------------
 -> [I] Dictionary Info:

Train len:  204000
Dimension:  128
Clusters:  200
Learning dictionary by Kmeans…
Done.

Step 3: Represent Dataset (BoVW)
--------------------------------------------------

Processing val set: 100%|      | 340/340 [00:03<00:00, 106.49it/s]
 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  200

Step 4: Running Experiment (Validation)
--------------------------------------------------

running the val phase: 100%|      | 340/340 [00:03<00:00, 99.62it/s]
Average accuracy in the validation set: 16.24%

--------------------------------------------------
Step 2: Dictionary Creation | K = 400
--------------------------------------------------
 -> [I] Dictionary Info:

Train len:  204000
Dimension:  128
Clusters:  400
Learning dictionary by Kmeans…
Done.

Step 3: Represent Dataset (BoVW)
--------------------------------------------------

Processing val set: 100%|      | 340/340 [00:03<00:00, 105.05it/s]
 -> [I] Space Describing Info:
```

```
Number of images:  340
Number of labels:  340
Dimension:  400

Step 4: Running Experiment (Validation)
--------------------------------------------------

running the val phase: 100%|      | 340/340 [00:03<00:00, 103.38it/s]

Average accuracy in the validation set: 15.76%

--------------------------------------------------
Step 2: Dictionary Creation | K = 600
--------------------------------------------------
 -> [I] Dictionary Info:

Train len:  204000
Dimension:  128
Clusters:  600
Learning dictionary by Kmeans…
Done.

Step 3: Represent Dataset (BoVW)
--------------------------------------------------

Processing val set: 100%|      | 340/340 [00:03<00:00, 104.47it/s]

 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  600

Step 4: Running Experiment (Validation)
--------------------------------------------------

running the val phase: 100%|      | 340/340 [00:03<00:00, 100.77it/s]

Average accuracy in the validation set: 15.41%

--------------------------------------------------
Step 2: Dictionary Creation | K = 800
--------------------------------------------------
 -> [I] Dictionary Info:

Train len:  204000
Dimension:  128
Clusters:  800
Learning dictionary by Kmeans…
Done.
```

```
Step 3: Represent Dataset (BoVW)
--------------------------------------------------

Processing val set: 100%|     | 340/340 [00:03<00:00, 104.27it/s]

 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  800

Step 4: Running Experiment (Validation)
--------------------------------------------------

running the val phase: 100%|     | 340/340 [00:03<00:00, 99.02it/s]

Average accuracy in the validation set: 14.71%

==================================================
ALGORITHM: GRID-SIFT
==================================================
--------------------------------------------------

Step 1: Vocabulary Creation
--------------------------------------------------

Processing train set: 100%|     | 680/680 [00:45<00:00, 14.96it/s]

 -> [I] Image Loader Info:

Train len:  272000
Number of images:  680
Descriptor size:  128

--------------------------------------------------
Step 2: Dictionary Creation | K = 100
--------------------------------------------------

 -> [I] Dictionary Info:

Train len:  272000
Dimension:  128
Clusters:  100
Learning dictionary by Kmeans…
Done.

Step 3: Represent Dataset (BoVW)
--------------------------------------------------

Processing val set: 100%|     | 340/340 [00:03<00:00, 102.54it/s]

 -> [I] Space Describing Info:
```

```
Number of images:  340
Number of labels:  340
Dimension:  100

Step 4: Running Experiment (Validation)
-------------------------------------------------

running the val phase: 100%|      | 340/340 [00:02<00:00, 114.09it/s]

Average accuracy in the validation set: 15.53%

-------------------------------------------------
Step 2: Dictionary Creation | K = 200
-------------------------------------------------
 -> [I] Dictionary Info:

Train len:  272000
Dimension:  128
Clusters:  200
Learning dictionary by Kmeans…
Done.

Step 3: Represent Dataset (BoVW)
-------------------------------------------------

Processing val set: 100%|      | 340/340 [00:03<00:00, 102.89it/s]

 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  200

Step 4: Running Experiment (Validation)
-------------------------------------------------

running the val phase: 100%|      | 340/340 [00:03<00:00, 110.47it/s]

Average accuracy in the validation set: 17.21%

-------------------------------------------------
Step 2: Dictionary Creation | K = 400
-------------------------------------------------
 -> [I] Dictionary Info:

Train len:  272000
Dimension:  128
Clusters:  400
Learning dictionary by Kmeans…
Done.
```

```
Step 3: Represent Dataset (BoVW)
--------------------------------------------------

Processing val set: 100%|     | 340/340 [00:03<00:00, 108.67it/s]

 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  400

Step 4: Running Experiment (Validation)
--------------------------------------------------

running the val phase: 100%|     | 340/340 [00:03<00:00, 106.40it/s]

Average accuracy in the validation set: 16.41%

--------------------------------------------------
Step 2: Dictionary Creation | K = 600
--------------------------------------------------
 -> [I] Dictionary Info:

Train len:  272000
Dimension:  128
Clusters:  600
Learning dictionary by Kmeans…
Done.

Step 3: Represent Dataset (BoVW)
--------------------------------------------------

Processing val set: 100%|     | 340/340 [00:03<00:00, 103.02it/s]

 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  600

Step 4: Running Experiment (Validation)
--------------------------------------------------

running the val phase: 100%|     | 340/340 [00:03<00:00, 96.02it/s]

Average accuracy in the validation set: 16.68%

--------------------------------------------------
Step 2: Dictionary Creation | K = 800
--------------------------------------------------
 -> [I] Dictionary Info:
```

```
Train len:  272000
Dimension:  128
Clusters:  800
Learning dictionary by Kmeans…
Done.

Step 3: Represent Dataset (BoVW)
------------------------------------------------

Processing val set: 100%|      | 340/340 [00:03<00:00, 95.50it/s]

 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  800

Step 4: Running Experiment (Validation)
------------------------------------------------

running the val phase: 100%|      | 340/340 [00:03<00:00, 90.34it/s]

Average accuracy in the validation set: 18.24%
```

[16]:
```
show_results_table(results)
print("\nDiscussion of results:")
```

```
=======================================================================
TABELA 1: BoVW Algorithms (different dictionary sizes)
=======================================================================

   Dic. Size  random-orb  grid-orb    orb  orb-hamming   sift  random-sift  \
0        100        3.56      9.41  15.76         6.74  19.97        14.71
1        200        3.26      8.62  15.56         7.85  21.38        16.24
2        400        2.71      8.79  15.35        11.79  22.06        15.76
3        600        2.15      8.26  14.56        13.68  21.94        15.41
4        800        1.71      9.47  14.15        13.09  22.09        14.71

   grid-sift
0      15.53
1      17.21
2      16.41
3      16.68
4      18.24

Best BoVW configuration: Algorithm = SIFT, K = 800 → 22.09%

=======================================================================
```

```
TABELA 2: LBP (global descriptor)
======================================================================

  Algorithm  Accuracy (%)
0       LBP          7.85


LBP Average Accuracy: 7.85%

Discussion of results:
```

[24]:
```python
# Best configuration on TEST SET: SIFT with 800 clusters
print("\n" + "="*70)
print("BEST CONFIGURATION TEST - SIFT with 800 clusters")
print("="*70)

algorithm = 'sift'
num_cluster = 800
dataset_path = '/Users/nicholasbarbosa/Mestrado/inf692/flowers_classes'

print(f"\nAlgorithm: {algorithm.upper()}")
print(f"Number of Clusters: {num_cluster}")
print(f"Dataset: TEST SET")
print("-"*70)

print("\nStep 1: Vocabulary Creation")
vocabulary = create_vocabulary(dataset_path, algorithm=algorithm)

print("\nStep 2: Dictionary Creation")
dictionary = create_dictionary_kmeans(vocabulary, num_cluster=num_cluster,␣
 ↪algorithm=algorithm)

print("\nStep 3: Represent Dataset (BoVW)")
space, labels = represent_dataset(dataset_path, dictionary,␣
 ↪algorithm=algorithm, test=True)

print("\nStep 4: Running Experiment on TEST SET")
acc_test = run_experiment(space, labels, dictionary, dataset_path,␣
 ↪algorithm=algorithm, test=True)

print("\n" + "="*70)
print(f"TEST SET ACCURACY (SIFT - 800 clusters): {acc_test:.2f}%")
print("="*70)
```

```
Python(4579) MallocStackLogging: can't turn off malloc stack logging because it
was not enabled.


======================================================================
```

```
BEST CONFIGURATION TEST - SIFT with 800 clusters
======================================================================

Algorithm: SIFT
Number of Clusters: 800
Dataset: TEST SET
----------------------------------------------------------------------

Step 1: Vocabulary Creation

Processing train set: 100%|      | 680/680 [07:21<00:00,  1.54it/s]

 -> [I] Image Loader Info:

Train len:  1233815
Number of images:  680
Descriptor size:  128

Step 2: Dictionary Creation
 -> [I] Dictionary Info:

Train len:  1233815
Dimension:  128
Clusters:  800
Learning dictionary by Kmeans…

Python(4912) MallocStackLogging: can't turn off malloc stack logging because it
was not enabled.

Done.

Step 3: Represent Dataset (BoVW)

Processing test set: 100%|      | 340/340 [00:17<00:00, 19.30it/s]

 -> [I] Space Describing Info:

Number of images:  340
Number of labels:  340
Dimension:  800

Step 4: Running Experiment on TEST SET

running the test phase: 100%|      | 340/340 [00:17<00:00, 19.20it/s]

Average accuracy in the test set: 23.50%

======================================================================
TEST SET ACCURACY (SIFT - 800 clusters): 23.50%
======================================================================
```

[27]:
```
print(algorithm)
retrieve_single_image( space, labels, dictionary, dataset_path ,␣
↪algorithm=algorithm)
```

```
sift
```


```
Accuracy for image id 694: 20.00%
```

```
/Users/nicholasbarbosa/Mestrado/inf692/flowers_classes/jpg/8/image_0694.jpg
Image: 694 with label 8
Closest image: 694 with distance 0.0 and label 8
Distances:  [[ 0.         60.8194048  60.89334939 61.2780548  61.31883887
61.59545438
  62.19324722 63.61603571 64.95382976 65.88626564 66.00757532]]
Indices:  [173  63  93  83  23 164 174  69 110  67  11]
Labels:  [8, 3, 4, 4, 1, 8, 8, 3, 5, 3, 0]
```

0.11 Discussion of results

Os resultados mostram que o SIFT foi superior em todos os tamanhos de dicionário, pois os descritores de 128 dimensões são mais discriminativos que os de 32 do ORB. O ORB-Hamming teve um pior desempenho que o ORB Euclidiano porque a conversão para binário perde precisão nos descritores float. Random e grid foram piores apenas para ORB, pois SIFT detecta keypoints mais robustos, mitigando a detecção artificial. Isso confirma que descritores robustos compensam detecções artificiais, enquanto conversões inadequadas prejudicam o desempenho.